Exceptions and Error Handling

Exceptions

- An *exception* is an event that occurs during the running of a program, that disrupts the normal flow of the program's instructions.
- Java allows programs to manage such errors by diverting control to special program code units that execute when the errors occur.
- These special code units are called *exception handlers*.
- The programmer must write these handlers.

Types of exceptions in Java

a. **Predefined Exceptions (Built-in-Exceptions):** Predefined exceptions are those exceptions that are already defined by Java system. These exceptions are also called built-in-exceptions.

The built-in-exception is further divided into two categories: checked and unchecked exceptions.

b. **Custom Exceptions:** Custom exceptions are those exceptions that are created by users or programmers according to their own needs. The custom exceptions are also called user-defined exceptions that are created by extending the exception class.

Examples of Exceptions

- Opening a non-existing file in your program.
- Reading a file from a disk but the file does exist there.
- Writing data to a disk but the disk is full or unformatted.
- When the program asks for user input and the user enters invalid data.
- When a user attempts to divide an integer value by zero, an exception occurs.
- When a data stream is in an invalid format, etc.

Examples of Exceptions

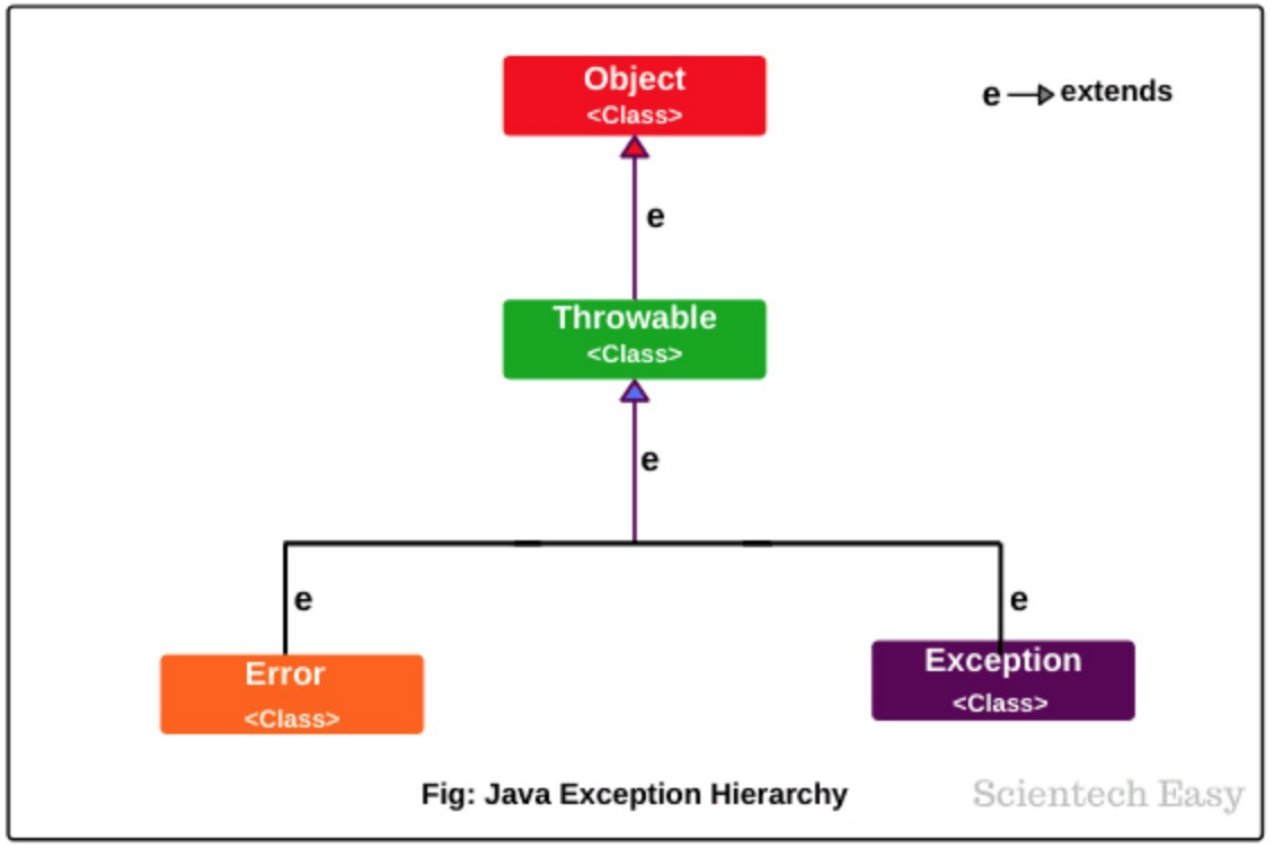
```
public class Test
{
public static void main(String[] args)
{
int a = 20;
int b = 30;
int c = 10;
int x = (a * b)/(a - b + c);
System.out.println("Result: " +x);
  }
}
```

```
public class Test
{
public static void main(String[] args)
{
int[ ] list = new int[4];
System.out.println(list[4]);
  }
}
```

Exception handling

- The mechanism of handling unexpected errors in a java program is called exception handling.
- It is a powerful mechanism to handle runtime errors, ClassNotFoundException, FileNotFoundException, IOException, etc. so that the normal execution flow of the program can be maintained.
- The code that catches the exception thrown by JVM is called **exception handle**r
    - The main advantage of exception handling technique is to maintain the normal flow of the program.
    - It provides flexibility in handling situations of errors.
    - It allows us to define a user-friendly message to handle the exception.
    - It helps to separate “Error-Handling code” from “Regular code.”

Exception Hierarchy



Throwable class is the parent class of all exception types. It is an immediate subclass of the Object class. Below Throwable class, there are two subclasses (two child objects**) Error and Exception.**

Errors vs Exceptions

Error: Error class is the subclass of Throwable class and a superclass of all the runtime error classes. It terminates the program if there is a problem related to a system or resources (JVM).

Exception: An exception is an abnormal condition that is caused by runtime error in the program. It is the superclass of all exceptions in Java. It is further divided into checked and unchecked (runtime) exceptions.

Errors vs Exceptions

Exception	Error
1. Exception represents the problem that can be solved.	1. Error represents the problem that cannot be solved.
2. Exceptions are related to the application.	2. Errors are related to the environment in which the application is running.
3. Exceptions can be handled and can continue the execution of program.	3. Errors cannot be handled.
4. Exceptions are further classified into checked and unchecked exceptions.	4. Errors are not further classified into such classification.

Exception Hierarchy

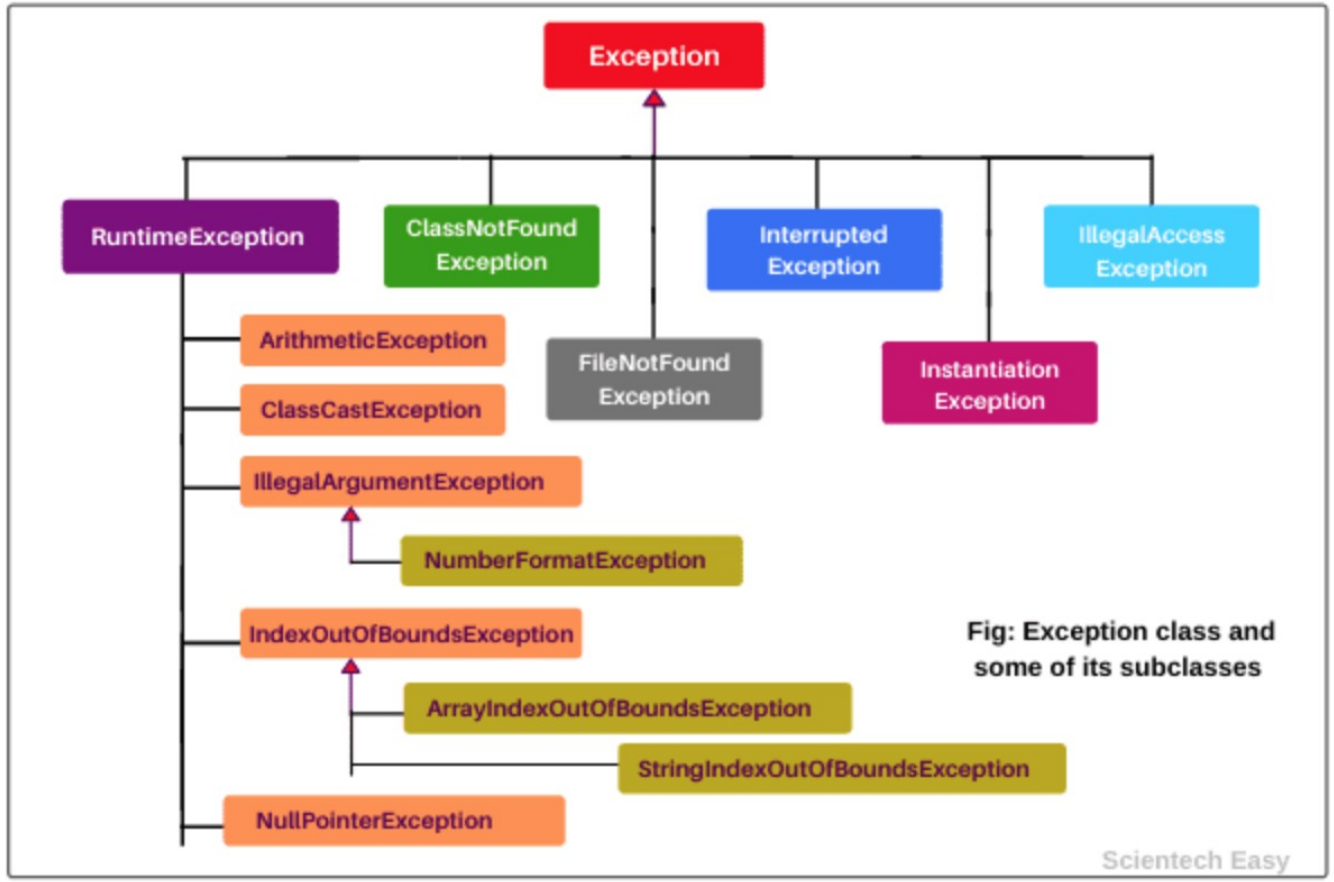


Fig: Exception class and some of its subclasses

Checked vs Unchecked Exceptions

Checked exceptions are those exceptions that are checked by Java compiler

- ClassNotFoundException
- InterruptedException
- InstantiationException
- IOException
- SQLException
- IllegalAccessException
- FileNotFoundException, etc

Runtime exceptions are those exceptions that are checked by JVM at

- ArithmeticException
- ClassCastException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NegativeArraySizeException
- ArrayStoreException
- IllegalThreadStateException
- SecurityException, etc.

How do we catch an exception?

We can catch an exception in either of two ways. They are:

- By try-catch block
- By using throws clause

Java provides five essential keywords to handle an exception. They are:

- try
- catch
- finally
- throw
- throws

Try-Catch Blocks

There are three possible forms of try block that are:

- try-catch
- try-finally
- try-catch-finally

A try is a block of code or statements that might throw an exception. That's why a try block is also known as exception generated block. An exception generated code (risky code) must be placed within a try block.

A catch is a block of code that handles the exception thrown by the try block. That's why it is also known as exception handler block.

Finally

A finally block is a block of code that is always executed whether an exception occurs within a try block or not.

finally block or clause is used for freeing up resources, cleaning up code, db closing connection, io stream, for terminating threads, etc.

```
try
{
    // Statements to be monitored for exceptions
}

//You can't keep statements here

catch(Exception ex)
{
    //Catching the exceptions here
}

//You can't keep statements here

finally
{
    // This block is always executed
}
```

Throw

Throw in Java is a keyword that is used to throw a built-in exception or a custom exception explicitly

```
public class Main {
  static void checkAge(int age) {
    if (age < 18) {
      throw new ArithmeticException("Access denied - You must be at least 18 years old.")
    }
    else {
      System.out.println("Access granted - You are old enough!");
    }
  }

  public static void main(String[] args) {
    checkAge(15); // Set age to 15 (which is below 18...)
  }
}
```

Throws

If a method is capable of throwing an exception that it could not handle, then it should specify that exception using throws keyword. It helps the callers of that method in handling that exception. The syntax for using throws keyword is,

```
return_type method_name(parameter_list) throws exception_list
{
    //some statements
}
```

•Multiple exceptions can be declared using throws keyword separated by commas.

```
static void methodWithThrows() throws NumberFormatException,
NullPointerException{

}
```

Throw vs Throws

Throw Clause	Throws Clause
1. throw clause is used when we want to throw an exception explicitly and want to handle it using catch block.	1. throws clause is used when we do not want to handle the exception and throw it out of the method.
2. Throw keyword is used inside the method body.	2. Throws keyword is used in method declaration (signature).
3. Throw is always followed by an object of Exception. e.g. throw new FileNotFoundException	3. Throws is always followed by name of exception class. e.g. throws FileNotFoundException
4. Throw is used to throw only one exception at a time.	4. Throws is used to throw multiple exceptions at a time. e.g. public void method()throws IOException, SQLException

Rethrowing

When an exception occurs in a try block, it is caught by a catch block inside the same method. The same exception object out from catch block can be rethrown explicitly using throw keyword. This mechanism is called rethrowing of exception in Java.

```
try {
            IOException ae = new IOException();
            throw ae;
        }
catch (IOException ae) {
            System.out.println("Error Printing 1");
            throw ae;
        }
```

Re-thrown exception must be handled some where in the program, otherwise program will terminate abruptly.

Some interesting questions

1. Can catch block throw an exception?

2. Can finally block throw an exception?

3. Can we throw multiple exceptions in one throw statement ?

4. As a programmer, when would you use try-catch and when would you use throw?